

Paragon Apartment Management System (PAMS)

Agile Development Report

Omar Hassanin 24062048

Ahmed 24062019

1. Introduction

The Paragon Apartment Management System (PAMS) is a desktop application created to help manage day-to-day apartment operations. It allows staff to handle tenants, record payments, track maintenance requests, and generate reports in one place. The system was developed to replace manual processes that can be slow and error-prone. By bringing everything together in a single platform, it makes the overall management process simpler and more efficient, while also improving organization and control across different locations.

2. Development Methodology (Agile)

The project was developed using an Agile approach, which focuses on building the system step by step rather than all at once. Instead of completing everything in a single phase, the work was divided into smaller stages, with each stage focusing on a specific feature or part of the system.

During each stage, the process usually involved understanding the requirements, designing the solution using UML diagrams, implementing the feature, and then testing it. This made it easier to track progress and fix problems early, rather than leaving everything until the end. It also allowed the system to improve gradually as new features were added and refined over time.

3. Development Process

3.1 Requirement Analysis

The system requirements were derived from the case study, focusing on key components such as:

- User management (multiple roles)
- Tenant management
- Apartment management
- Payment and billing
- Maintenance handling
- Report generation

3.2 System Design

The system was designed using UML diagrams:

- Use Case Diagram to represent system functionality
- Class Diagram to define system structure
- Sequence Diagrams to illustrate system interactions
-

3.3 Implementation

The system was built using Python and includes a graphical user interface to make it easier to use. A database is used in the background to store and manage all the important information, such as tenant details, apartment records, payments, and maintenance requests. This allows the system to keep everything organised and ensures that data can be accessed and updated when needed.

Key features implemented include:

- User login and role-based access
- Tenant registration and management
- Apartment allocation
- Payment recording
- Maintenance request tracking
- Report generation

3.4 Testing

Testing was performed using both manual and structured test cases. Each system feature was tested with valid and invalid inputs to ensure reliability.

Testing included:

- Functional testing (checking system features)
- Input validation testing
- Error handling

4. Contribution

Each group member contributed to different parts of the system throughout the project. The work was shared across the team, including designing the UML diagrams such as the use case, class, and sequence diagrams. Members were also involved in implementing the core system features, developing different parts of the user interface, and carrying out testing to make sure everything worked as expected. This collaborative approach helped ensure that all aspects of the system were completed and properly integrated.

5. Security Considerations

The system includes some basic security features to help protect user data. For example, users are required to log in before accessing the system, and different roles are given different levels of access. This means that only authorised users can view or manage certain information. Overall, these measures help ensure that sensitive data is protected and that users can only perform actions relevant to their role.

6. Conclusion

Using an Agile approach helped the team develop the system in a more flexible and manageable way, as it allowed features to be built and improved step by step while testing along the way. The final system covers the main requirements from the case study and works as a practical solution for managing apartment operations.

In conclusion, the project showed us how to have a good understanding of object-oriented design, how to work with databases, and how to apply software development practices in a real system.